Homework #3 (HW3)

Autumn 2026 - GPU Computing

DUE: 6 November 2026, 11:59 PM

Learning Objectives

- Get experience understanding control divergence in warps
- Implement and profile kernels with warp divergence behavior
- Implement improvements to the MMULT tiled-phased algorithm kernel
- Get a better understanding of kernel launch configuration tuning.
- Practice using Nsight Compute GUI for profiling kernel execution and analyzing kernel performance issues and optimization.

Questions

Please turn in written (or typed) solutions to these problems.
In order to receive credit you must show your work and/or describe your reasoning/explanation for the selected answer.

I. If the SM of a CUDA device can take up to 1536 threads and up to 4 thread blocks. Which of the following block configurations would result in the largest number of threads in the SM?
   A. 128 threads per block
   B. 256 threads per block
   C. 512 threads per block
   D. 1024 threads per block

II. For a vector addition, assume that the vector length is 2000, each thread calculates one output element, and the thread block size is 512 threads. How many threads will be in the grid?
   A. 2000
   B. 2024
   C. 2048
   D. 2096

III. With reference to the previous question, how many warps do you expect to have divergence due to the boundary check on vector length?
   A. 1
   B. 2
   C. 3
   D. 6

IV. Assume a device that allows up to 64 blocks per SM and 2048 threads per SM. Indicate which of the following assignments per SM are possible. IN the cases in which it is possible, indicate the occupancy level.
    A. 8 blocks with 128 threads each
    B. 16 blocks with 64 threads each
    C. 32 blocks with 32 threads each
    D. 64 blocks with 32 threads each
    E. 32 blocks with 64 threads each

V. Consider a GPU with the following hardware limits: 2048 threads per SM, 32 blocks per SM, and 64K (65,536) registers per SM. **For each of the following kernel characteristics**, specify whether the kernel can achieve full occupancy. If not, specify the limiting factor, and the maximum occupancy % achieved.
    A. kernel uses 128 threads per block, and 30 registers per thread.
    B. kernel uses 32 threads per block, and 29 registers per thread.
    C. kernel uses 256 threads per block, and 34 registers per thread.

Analysis

Control Divergence

Calculate the impact of control divergence for the N tiles in the tiled MMULT algorithm

- In the same way as demonstrated in Lecture 4 for the M tiles
    a. refer to lecture 4 slides for details
    b. Also see PMPP section 4.5
- Discuss the behavior of the Type 1 and Type 2 blocks
    a. How many warp phases have divergence for each?
    b. What is the expected performance impact %?
        - for each Type (1/2)?
        - and the combined Total?
- Q: When will the warp divergence impact become minimal?
- Q: How does varying tile size affect arithmetic intensity?

Programming Exercises

NOTE: Always use a RELEASE (x64) build when profiling your code with Nsight Compute (NCU) and remove all printfs! Otherwise profiling results will be incorrect.

Control Divergence

- After completing the Analysis-Control Divergence section above, now implement simple CUDA kernels which exhibit control divergence behavior for each of the following cases (as discussed in Lecture 4). The kernels don't need to do any significant calculations or produce any particular result values - we are just interested in observing the warp divergence effects. *However, be sure to at least update an output result variable to GMEM in each branch case - otherwise the compiler may optimize and remove the whole branch if it never actually does anything!*
    1. single branch (control flow bifurcation)
    2. nested branch scenarios (at least three different control paths per warp)
- use Nsight Compute to profile the kernels for cases with/without control divergence
    a. Refer to Lecture 4 demo of Nsight Compute for overview of profiling w NCU
    b. Be sure to provide appropriate input test data grids to your kernels that will correctly and predictably cause the kernel behavior to be either divergent, or non-divergent, as desired.
        - A possible approach is to provide a kernel input argument which will make the kernel take a divergent versus non-diverging code branch.
    c. Also be sure your kernel executes over a reasonable number of threads and blocks in the full grid, otherwise the NCU metrics may not be accurate.
    d. Save the Nsight Compute kernel profile reports (*.ncu-rep) for executions of each of the diverging kernels for the cases
        - with divergence
        - without divergence
    e. Based on the configuration of your kernels for control divergence, can you explain how the Nsight Compute metrics can capture or identify when divergence is happening?
        - Look at
            - Occupancy
                - Theoretical vs Achieved Occupancy
                - Theoretical vs Achieved Active Warps Per SM
            - Warp State Statistics
                - Avg. Active Threads Per warp
                - Avg. Not Predicated Off Threads Per Warp

- Can you find any other general GPU profiling metrics (GPU Speed of Light) which change between divergent and non-divergent runs?

MMULT Variants

- Implement the following MMULT algorithm variants
  - CPU serial reference version - for comparison and verification
  - GPU MMULT kernel v0 (naive, all-GMEM)
  - GPU general shared-memory tiled/phased kernel with boundary checks
    - support general nonsquare conforming input matrix dimensions
      1. P: j x l, M: j x k, N: k x l
    - support tile sizes (= thread block) which are not perfect multiple of input matrix dims
    - (refer to PMPP Ch 5.4 - 5.5 and lecture 3/4 slides which discuss these phased-tiled algorithms and generalized techniques)
- Provide host verification code that checks all three kernels produce consistent results, within an acceptable error tolerance threshold.
  - what is the maximum single-element abs error fraction observed?
    - absolute error fraction = abs[Pcpu(x,y) - Gpu(x,y)] / Pcpu(x,y)
    - evaluate for several input/output matrix sizes:
      1. P : 100x100, k = 100
      2. P : 1000x1000, k=1000
      3. P : 2000x3000, k = 2500
- For each of the 3 input/output matrix size cases above, Compile your host application and kernels using a Release Build configuration in Visual Studio. Now use Nsight Compute to generate profile reports. View the Roofline chart in the GPU Speed of Light section.
  - Use baselines to add a baseline for each case, so that all the kernel variants can be viewed on a single Roofline chart.
- Experiment with different TILE_WIDTH sizes (try 8, 12, 16, 32)
  - and also different thread-block launch configurations (keeping within the resource limitations of your specific GPU device!) *NOTE that in the current kernel implementations we've considered we assume TILE_WIDTH dimension is the same as the thread-block dimensions (both square)*
    - (8, 8, 1)
    - (12, 12, 1)
    - (16, 16, 1)
    - (32, 32, 1)
  - Using baselines, try to put everything onto a single Roofline plot.
    - Note- each baseline can be renamed with a more meaningful name.
  - Which configurations (TILE_WIDTH + threadblock size) provide the best performance?

        - highest AI?
        - Highest compute throughput?
        - Fastest execution Duration?
        - Highest Occupancy?
        - Under any configurations or input sizes are kernels in the compute-bound regime?
    - Do these results vary with the size of the input/output matrices?
        - Try to explain the behavior in a few sentences, referring to available device resources and other Nsight profile metrics as applicable.
- Select the best performing configuration for the case with P: 2000x3000, k = 2500.
    - In your VS Project Properties -> Configuration Properties -> CUDA C/C++ ->Command Line
        - In Additional Options: enter the compiler option `--fmad=false`
            1. this disables use of the fused instruction equivalent (FFMA)
            2. it should be defaulted to 'true'
        - click OK
    - Rebuild a Release version of your code and profile it with Nsight Compute.
    - Did the performance change?
        - Consider all the metrics: AI, execution duration, compute throughput, occupancy?
    - When you disable fused multiply add, you force the hardware to split a single FMA into separate MUL and ADD instructions. How does it impact arithmetic intensity?
- ADDITIONAL QUESTIONS
    - What if we transpose the input matrix N, so that both the matrices are row-major in the memory, is the performance overhead required for doing the transpose worth it?
    - If you push the TILE_WIDTH to 32, you max out the hardware limit of 1024 threads per block. Does maximizing threads per block guarantee the highest occupancy?
    -

SUBMIT

1. Submit answers to the questions, including shown work and explained reasoning.
    a. Including answers to the control divergence analysis questions
2. Submit your CUDA host application source code
    a. .cu, .cpp and .h files only - do not submit all project & solution files and other build by-products!
3. Submit your CUDA kernel functions
    a. may be in the same file as your host code if using a single *.cu file
4. Submit your NCU profiling results for all test cases and global + shared kernels

    a. also include your calculated kernel execution time and kernel instruction intensity values.
5. Submit a page (PDF or Word) with a discussion of your profiling analysis for the MMULT kernels and the various launch configurations and input/output matrix sizes, and the FMAD option.
    a. include (screenshots) of the result Nsight Compute profile reports with the set of profile baselines showing all the various configurations tested.
    b. Include roofline plots for the different kernels and different input sizes, etc.